

A Novel Architecture For DECIMAL CONVERSION In 8-BIT MCU

K.M. ABUBEKER, REEJU ELISA BABY AND AJAI MATHEW

This article highlights common errors in 8-bit instruction sets, and introduces a novel architecture for decimal conversion instructions, especially in 8051, 8086, 8085 and PIC microcontrollers (MCUs). Statistical analysis of different methodologies has revealed that decimal conversion instructions in microprocessors and MCUs do not offer an error-free process. Bugs can be common for the above-mentioned MCUs. The algorithms discussed in this article have been tested for various platforms and offer accurate results on all 8-bit MCUs and microprocessors.

Binary-coded decimal number system

Binary-coded decimal (BCD) numbers are used to represent digital numbers, that is, zero to nine. In computer language, there are two terms used for BCD numbers: unpacked and packed. These two are shown in Fig. 1.

In another example, number 74 in packed BCD format would be 0111 0100, whereas, unpacked BCD format would be 0000 0111 and 0000 0100.

Unpacked BCD. The lower four

bits of the numbers represent BCD number. Only one byte of memory is required for unpacked BCD numbers or 8-bit register to contain it. In unpacked BCD numbers, upper bits are zeros (0s). For example, 0000 0001, 0000 0010, up to 0000 1001 are unpacked BCD numbers for 1, 2, ... 9, respectively.

Packed BCD. In packed BCD numbers, a single byte has two BCD numbers, one in the lower four bits and the other in the upper four bits. Both lower and higher four bits are used. Only one byte of memory is required to store packed BCD operands. For example, 0101 0010 is packed BCD for 52H.

While adding two BCD numbers, the result must be a BCD number. If that is not the case, then it must be corrected by adding number 6 to it.

Computers use hexadecimal number system. If you add two BCD numbers 9 and 1, the result should be 10. But a microprocessor uses hexadecimal addition and gives the result as A instead of 10. Here, A is not a valid BCD number (BCD numbers are from 0 to 9).

To solve the problem, add six to

the hexadecimal number system (A, B, C, D, E, F). This is done by adding number 6 to the result to get the expected result. Consider the following examples to know the BCD conversion process, as given in Table II.

In the first example, lower and upper nibbles of the sum of 10 and 20 lie in the range of 1 to 9. So, in this case, conversion procedure is not required.

In the second case, sum of 15 and 20 is 3A. Lower nibble (A) of the result is not in the prescribed range (0000 0001, 0000 0010, up to 0000 1001). To obtain the BCD result, add number 6 to the result. The required result is 40.

The same can be applicable to the third example, too.

DA instruction

DA, or decimal adjust, is an instruction used in 8051 MCUs for DA operation after addition operation like ADD A, R0, ADC A, R1, SUB A, R0, SUBB A, R1, etc. DA for addition instruction is designed to correct BCD addition problems. Like the decimal conversion discussed earlier, here too DA instruction adds number 6 to either lower or higher nibble, if needed. For detailed analysis, necessary conditions and procedures in DA instruction are explained below.

Working of DA instruction.

1. DA instruction is effective only after arithmetic operation.

2. If lower nibble of result is greater than 9, or if auxiliary carry is 1 (AC = 1), add 0110 in binary or 6H in the lower nibble to correct BCD problems.

3. If upper nibble of result is greater than 9, or if carry is 1 (CY = 1), add 0110 in binary or 6H in the upper nibble to correct BCD problems.

TABLE I BCD NUMBERS FOR 0 THROUGH 9	
Decimal Number	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Decimal :	9	1
Binary :	0000 1001	0000 0001
Unpacked BCD		
Decimal :	9	1
Binary :	1001 0001	
Packed BCD		

Fig. 1: Unpacked and packed BCD

TABLE II PACKED BCD EXAMPLES		
Example 1	Example 2	Example 3
10 + 20 = 30d	15 + 25 = 40d	15 + 55 = 70d
0001 0000 + 0010 0000 0011 0000 (30d)	0001 0101 + 0010 0101 0011 1010 + (3A) 0110 0100 0000 (40d)	0001 0101 0101 0101 0110 1010 + (6A) 0110 0111 0000 (70d)